

# Review Score Prediction from Amazon Music Reviews

---

## Preliminary Report

Deniz Yalim Yilmaz & Aleks Slucz  
May 2026

### 1. Project Overview

---

This project sets out to answer a simple question: can a machine learning model read an Amazon product review and correctly guess the star rating? The model takes only the written review text as input and outputs a predicted score from 1 to 5.

Three classification algorithms will be trained on the same dataset and then compared against each other. The goal is to find out which one handles this kind of text-to-score task best.

#### Research Goals

| Goal      | Description                                                                                           |
|-----------|-------------------------------------------------------------------------------------------------------|
| Primary   | Compare Naive Bayes, Logistic Regression, and Linear SVM on predicting star ratings from review text. |
| Secondary | Test how well a model trained on instrument reviews handles reviews from other product categories.    |
| Optional  | See whether using short review summaries instead of full text changes model accuracy.                 |

#### Tech Stack

| Tool            | Role                                                |
|-----------------|-----------------------------------------------------|
| Python          | Core language for all model building and evaluation |
| pandas          | Dataset loading, cleaning, and manipulation         |
| scikit-learn    | TF-IDF vectorization and ML algorithm training      |
| GitHub          | Version control and code repository                 |
| Microsoft Teams | Team communication                                  |

### 2. Dataset

---

The dataset is the Amazon Musical Instruments Reviews dataset from Kaggle. It contains customer reviews across 9 columns. Only two of those columns are directly needed for training.

## 2.1 Column Overview

| Column name    | Description                      | Datatype     | Notes                                                            |
|----------------|----------------------------------|--------------|------------------------------------------------------------------|
| reviewerID     | ID of the reviewer               | string       | Not relevant for this project                                    |
| asin           | ID of the product being reviewed | String       | Not relevant for this project                                    |
| reviewerName   | Name of the reviewer             | String       | Not relevant for this project                                    |
| helpful        | Helpfulness score of the review  | integer List | Omitting reviews with low helpfulness rating might be considered |
| reviewText     | Review given by the reviewer     | String       | Critical column                                                  |
| overall        | Final score given from 1 to 5.   | int          | Critical column                                                  |
| summary        | Summary of                       | String       | Might be considered further                                      |
| unixReviewTime | Time of review in unix time      | int          | Not relevant for this project                                    |
| reviewTime     | Time of review.                  | String       | Not relevant for this project                                    |

## 2.2 Preprocessing Steps

Before training, the raw data needs to be cleaned and converted into a format the algorithms can work with. The steps below are applied in order.

| Steps                      | Action                                                      | Reason                                                                             |
|----------------------------|-------------------------------------------------------------|------------------------------------------------------------------------------------|
| Removing unrelated columns | Removing columns other than reviewText, overall, summarized | The rest of the columns are not helpful for this project.                          |
| Missing values             | Remove rows where reviewText or overall is missing.         | A model cannot learn from examples without text or labels.                         |
| Duplicates                 | Remove duplicated reviewText examples if present.           | Prevents repeated identical reviews from appearing in both training and test sets. |
| Label conversion           | Convert overall values to integer labels 1-5.               | Ensures consistent class labels for all algorithms.                                |
| Text normalization         | Lowercase text and strip whitespace.                        | Reduces unnecessary variation between reviews.                                     |
| TF-IDF vectorization       | reviewText should be converted to TF-IDF vectors            | Algorithms used cannot work with plain String input.                               |

## 2.3 Dataset Split and Class Balance

The dataset will be split into three parts: 70% for training, 15% for validation, and 15% for testing. A fixed random seed is used so that every algorithm gets exactly the same data split, keeping comparisons fair.

Product review datasets tend to have far more 4 and 5 star ratings than 1 or 2 star ratings. The class distribution will be checked before training. Stratified splitting will be used to keep that distribution consistent across all three sets. Undersampling and oversampling will also be tested to see if they improve results.

| Split      | Share | Purpose                                            |
|------------|-------|----------------------------------------------------|
| Training   | 70%   | Used to fit each model                             |
| Validation | 15%   | Used to tune and compare models during development |
| Test       | 15%   | Used only at the end to report final results       |

### 3. Algorithms

Three well-known classification algorithms were chosen. All three will receive the same TF-IDF vectors as input so results are directly comparable.

#### 3.1 Algorithm Summary

| Algorithm                     | Reasoning                                                           | Expected Performance                                            |
|-------------------------------|---------------------------------------------------------------------|-----------------------------------------------------------------|
| Multinomial Naive Bayes       | Simple and fast to implement, good baseline algorithm.              | Expected to perform worse than others.                          |
| Logistic Regression           | Strong linear classifier.                                           | Expected to perform better than Naive Bayes but worse than SVM. |
| Linear Support Vector Machine | Chosen because SVMs are famous for performing great on text inputs. | Expected to perform the best out of all the algorithms.         |

#### 3.2 Why TF-IDF?

None of the three algorithms can work with raw text. TF-IDF (Term Frequency-Inverse Document Frequency) converts each review into a numerical vector. Each position in that vector represents a word or phrase, and its value reflects how important that word is in this review compared to the rest of the dataset. This helps prevent very common words like 'the' or 'and' from drowning out more meaningful terms.

### 4. Evaluation

Model performance will be measured using two metrics. Accuracy alone is not enough for a five-class problem, so Mean Absolute Error is included to show how far off the wrong predictions tend to be.

| Metric   | What it measures                                                 | Why it matters                         |
|----------|------------------------------------------------------------------|----------------------------------------|
| Accuracy | Percentage of reviews where the predicted score matches exactly. | Simple overall measure of correctness. |

| Metric                    | What it measures                                              | Why it matters                               |
|---------------------------|---------------------------------------------------------------|----------------------------------------------|
| Mean Absolute Error (MAE) | Average gap between the predicted score and the actual score. | Penalises large misses more than small ones. |

Test Scenarios

| Scenario     | Description                                                                 |
|--------------|-----------------------------------------------------------------------------|
| A (Main)     | All three models trained and evaluated on the musical instruments dataset.  |
| B (Summary)  | Models retrained using the summary column instead of the full review text.  |
| C (Optional) | Models from Scenario A tested on reviews from a different product category. |

Project Outputs

| Output            | Description                                              |
|-------------------|----------------------------------------------------------|
| GitHub Repository | Full source code, notebooks, and saved model files       |
| Results Tables    | Accuracy and MAE scores for every algorithm and scenario |
| Trained Models    | Saved model files for all three algorithms               |
| Final Report      | Written analysis of findings and conclusions             |

---

References

[1] P. Skiers, based on work by F. Szatkowski, "Project 10: Review score prediction," Introduction to Artificial Intelligence, May 2026.